

DEPARTMENT OF MANAGEMENT SCIENCE

MSCI 222: OPTIMISATION GROUP COURSEWORK EXERCISE

Coursework should be submitted online via Moodle by **2pm on Tuesday 23 January 2024**. You should submit **a report and four** (either Python or Macro-Enabled MS Excel, depending on the workshop you attended) **files that contain your algorithm codes for each question**. You should also **include the codes of your algorithms in the appendix of your reports**.

The exercise must be done as group work. Late work or evidence of collusion with other groups will be penalised in the manner specified in the department's teaching code of practice.

You should **describe every change in code if you are using the code examples from the computer workshops or new algorithm in detail**. You can describe the changes in detail in your reports and add comments to your codes that you are submitting with your report. Furthermore, you can also use the **example instances** to describe each step of new algorithms you develop.

Discuss the experimentation process by presenting the results from several runs for different data sets **with various parameters**. Describe in detail what experiments did you conduct and how did you make sure that how they are improving the performance of your algorithms for different data sets.

Up to 15% of your mark will be awarded by evaluating the performance of your algorithms on different instances (some will be shared in Moodle). For this purpose, **your code should run no more than 30 seconds and after 30 seconds, it should provide a feasible solution**. These feasible solutions should be including all cities in the data set and should return the right cost of the tour provided as an answer.

A set of data files are provided in Moodle. **The first line of every data file contains the number of cities** that this instance contains. **Every line** in the data files after this line contains **the IDs of the cities** (zero based list), **their x and y coordinates**. Your code should read every city coordinates one by one properly. The codes shared with the workshop can be used for this task.

The Travelling Salesperson Problem (TSP) is an optimisation problem. The TSP takes a set of cities and their coordinates (or the distance between every pair of cities) as input and **aims to find the tour that visits all cities with minimal cost/distance**. It is an NP-hard combinatorial optimisation problem, and finding and proving optimal solutions for large instances may take days or even years. **We have already developed a constructive and a local search algorithm for the problem during the workshops. You can use these codes as part of your coursework, with changes if needed.**

Question 1 (30 marks)

One of the heuristic approaches used to solve optimisation problems is the steepest descent (i.e. descent for minimisation, ascent for maximisation problems) method. This method requires an initial solution to start, so to use the steepest descent method, one should start with a constructive algorithm. Then, the steepest descent method would start from the current solution and checks all solutions in the neighbourhood and move to the best solution that improves the current solution. If it is not improving the current solution, it stays in the current solution and run again with a different set of neighbourhoods.

Write a code that solves the existing problem using a constructive and a steepest descent method you choose. You are allowed to use the codes of given constructive **nearest neighbour first** and **move city to best location** local search algorithm provided with the workshop. Then experiment with your code and provide how does

your algorithm perform (can be done by reporting the best cost found so far and percent improvement of the best cost in every iteration) for the instances provided given the number of iterations and running time.

Question 2 (30 marks)

Develop **one more constructive** and **two more local search** algorithms. One of the local search algorithms can use one of the move city algorithm's code we developed in the workshops as the base and change parts of it to create a new local search algorithm, but the other algorithm should use a different type of neighbourhood (e.g. swapping two cities, two-opt, three-opt). Developed algorithms should return feasible solutions with the cost calculated accurately. In your codes, **call the two constructive algorithms to generate two different solutions at the beginning of the heuristic and continue with the solution which is better** (i.e. lower cost). Then, **call your local search algorithms one after the other in every iteration**. You can either call one of them in every iteration (in order) or call all three of them in every iteration and choose the answer with the best solution. **Describe your new constructive and local search algorithms in your own words**. Run experiments like you do in Question 1. In addition to performance of the algorithm, also report the individual performance of each constructive and local search heuristics (e.g. ratio of times they perform better than the others, ratio of times they improve the current solution, etc.).

Question 3 (20 marks)

When there are multiple local search algorithms developed for the same problem, one of the approaches that works quite well is **the Variable Neighbourhood Search (VNS)**. In variable neighbourhood search, there is an **internal process that chooses better-performing neighbourhoods with higher probabilities**. However, it is also important to make sure that **none of the local search algorithms' selection probability is becoming zero**.

For this question, your heuristic should start with the same probability for the three local search algorithms. Then **every time an algorithm is improving the best solution, the heuristic should increase the better performing local search algorithm's selection probability**. If the probability of the algorithm that found a better solution is p , increase the probability by $0.1 \times (1 - p)$ and decrease the probabilities of the other methods by the same amount in total to make the sum of selection probabilities of the algorithms 1.

Here is an example showing how you should update your probabilities: If you have three algorithms, A1, A2 and A3, they will start with the same probability, $p_1 = p_2 = p_3 = 1/3 = 0.333$ at the beginning of your heuristics. Then you should generate a random number and check what range it falls into. If it is between 0 and p_1 , then A1 is selected as the next local search algorithm. If the probability is between p_1 and $p_1 + p_2$, then A2 is selected as the next local search algorithm. If the probability is between $p_1 + p_2$ and 1, then A3 is selected as the next local search algorithm. Let's assume the first random number generated is 0.65 (it would be selected randomly in your algorithms so would be different) so your algorithm should choose A2 since it falls between p_1 and $p_1 + p_2$. Then, if A2 improves the best solution by finding a solution better than the best solution so far before the call of A2, update the probability of A2 by $0.1 \times (1 - 0.333) = 0.067$. This should make $p_2 = 0.333 + 0.067 = 0.4$, $p_1 = 0.333 - 0.067/2 = 0.3$ and $p_3 = 0.333 - 0.067/2 = 0.3$ for the next iteration. Continue updating the selection of your local search algorithms (when you improve the best solution found so far) and choose the algorithm with a random number until the end of the algorithm.

Run experiments like you do in Question 2. **Your code should use a single constructive and 3 local search algorithms. The algorithm should start with the constructive algorithm. Then it should run one of the local search algorithms randomly in every iteration. The probability of selecting all three local search algorithms**

should be updated when one of them finds a solution which improves the current best solution as described above. You should also compare your run results with the runs in Question 2 considering different running times and iteration counts.

Question 4 (20 marks)

In the steepest descent algorithm, the current solution is updated only if the solution is improved by the next neighbourhood. However, this may result in being stuck in a local optimum. We may prevent this by using an algorithm that moves to inferior solutions with some probability in a structured way. In this question, you should do this by using the **Simulated Annealing (SA)** metaheuristic.

SA has two additional parameters: **Initial temperature, T_0 and the rate of cooling, α** (in some examples, there may also be an updating frequency parameter, but we can discard this for this activity if you do not want to use). **We start the algorithm by setting t , the temperature, to T_0 . Then, in every iteration, t is updated by multiplying by α .** In every iteration, if the new solution is improving the current solution, your algorithm should move to the new solution which is the same as the steepest descent. However, different from the steepest descent, **if the new solution is not improving the current solution, you should calculate Δ which is the difference between the current solution and the new solution** (please note Δ should always have a non-negative value). Then calculate the **acceptance probability $e^{-\Delta/t}$** . You should generate a random number between 0 and 1 and update your current solution if the random number you generated is less than the acceptance probability. Please also note, **since the current solution is not always the best solution so far in SA, the best solution should be kept in a separate field in the code.**

You should improve your code from Question 3 by **applying simulated annealing to your algorithm**. You should also **conduct a few experiments with different initial temperatures (T_0) and the rates of cooling (α)** to find parameters that works best for the TSP instances you are solving. A good initial temperature should allow the algorithm to move more inferior solutions at the beginning of the algorithm, but this should happen very rarely towards the end of the algorithm. **The rate of cooling should be strictly between 0 and 1**, close to 1.

Run experiments like you do in Question 3. Also compare your run results with the runs in Question 3 for different running times and iteration counts.